2. Expressions, Operators, and Type Conversions

02. Expressions, Operators, and Type Conversions

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `02-expressions-operators`

Learning objectives

- Use arithmetic, comparison, and logical operators correctly
- Explain why Go forbids implicit numeric conversions
- Convert between types safely and consciously
- Avoid integer division surprises

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Operators

Arithmetic: `+ - * / %` Comparison: `== != < <= > >=` Logical: `&& || !` Bitwise: `& | ^ << >>` (useful in systems code)

```
a, b := 7, 2
fmt.Println(a / b)   // 3  — integer division truncates
fmt.Println(float64(a) / float64(b)) // 3.5
```

Explicit conversions only

Unlike Python (or C in some cases), Go never silently converts `int` to `float64`:

```
var i int = 3
var f float64 = float64(i) // required
```

This is deliberate: silent conversions hide bugs in money, indices, and APIs.

Precedence and clarity

When in doubt, use parentheses. Clarity beats cleverness:

```
ok := (score >= 50 && attempts < 3) || admin
```

Common mistakes (especially from Python instincts)

- Integer division truncating when you expected a float (Python 3 does float `/`)
- Trying `int + float64` without conversion
- Using `=` vs `==` confusion (assignment vs comparison)

Exercises

Work in `exercises/02-expressions-operators/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/02-expressions-operators/` until you have an honest failing attempt.

Self-check

1. What does `7/2` evaluate to in Go?
2. Why must you write `float64(x)`?
3. Give an example where implicit conversion would be dangerous

Further reading

- https://go.dev/ref/spec#Operators
- https://go.dev/ref/spec#Conversions